

Theory of Programming Languages

6. Subtype

Kihong Heo



Motivation

- Relation between types
 - E.g, $\text{Dog} \subseteq \text{Animal}$,
- Generic programming: one code, many uses (polymorphism)
 - E.g., `printName(student)`, `printName(prof)`, `printName(visitor)`
- Introduced in Simula (1962)
 - The first object-oriented programming language
- Foundation of OOP languages
- Applied in many languages

Subtype in OCaml

```
type parent = [ `A | `B ]
type child = [ parent | `C ]

let child_care (x : child) =
  match x with `A -> "A" | `B -> "B" | `C -> "C"

let parent_care (x : parent) =
  match x with `A -> "A" | `B -> "B"

(* Find type error *)
let () =
  print_endline (child_care `A);
  print_endline (child_care `B);
  print_endline (child_care `C);
  print_endline (parent_care `A);
  print_endline (parent_care `B);
  print_endline (parent_care `C)
```

```
let f = function `On -> 1 | `Off -> 0 | `Number n -> n
(* val f : [< `Number of int | `Off | `On ] -> int = <fun> *)
(* The function can accept "less" (<) variants than the ones
specified. *)
```

```
let g = function `A -> `C | `B -> `D | x -> x
(* val g : ([> `A | `B | `C | `D ] as 'a) -> 'a = <fun> *)
(* The function can accept "more" (>) variants than the ones
specified. *)
```

Problem in STLC

- Too rigid typing rules
- Example

$$\frac{\Gamma \vdash E_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash E_2 : \tau_1}{\Gamma \vdash E_1 E_2 : \tau_2}$$

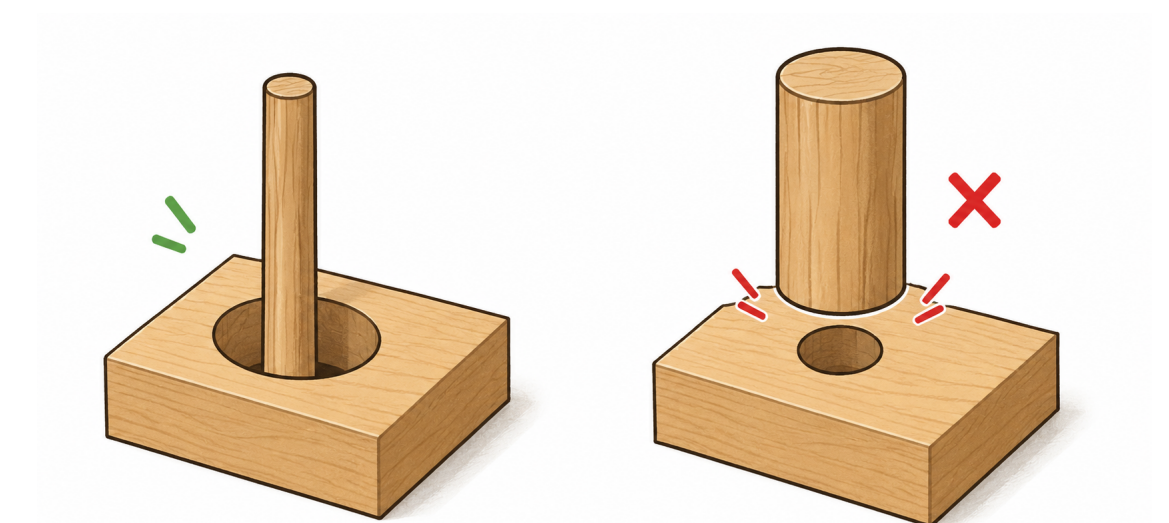
$(\lambda r : \{x : \text{nat}\}. r.x) \{x=0, y=1\}$

- How to make type systems accept this program?

Subsumption

Principle of safe substitution. τ_1 is a subtype of τ_2 , written $\tau_1 <: \tau_2$, means that any term of type τ_1 can be safely used in a context where a term of type τ_2 is expected.

- Rule of subsumption
$$\frac{\Gamma \vdash E : \tau_1 \quad \tau_1 <: \tau_2}{\Gamma \vdash E : \tau_2}$$
- Example: $\{x : \text{nat}, y : \text{nat}\} <: \{x : \text{nat}\}$



Less is More (과유불급)

Subtype (1)

- Language $\lambda_{<}$: Simply typed λ -calculus with subtyping
- Primitive types: nat, int
- Compositional types: function, record, variant
- Typing rule: same as before, except the subsumption rule

$$\begin{array}{l}
 E \rightarrow \dots \mid n \mid x \mid \lambda x:\tau.E \mid E \ E \\
 \quad \mid \{l_i = E_i \mid i \in 1 \dots n\} \mid E.l \\
 \quad \mid l \ E \text{ as } \tau \mid \text{case } E \text{ of } l_i \ x_i \Rightarrow E_i \mid i \in 1 \dots n \\
 \\
 \tau \rightarrow \text{nat} \mid \text{int} \mid \tau \rightarrow \tau \\
 \quad \mid \{l_i : \tau_i \mid i \in 1 \dots n\} \\
 \quad \mid \langle l_i : \tau_i \mid i \in 1 \dots n \rangle
 \end{array}
 \qquad
 \frac{\Gamma \vdash E : \tau_1 \qquad \tau_1 <: \tau_2}{\Gamma \vdash E : \tau_2}$$

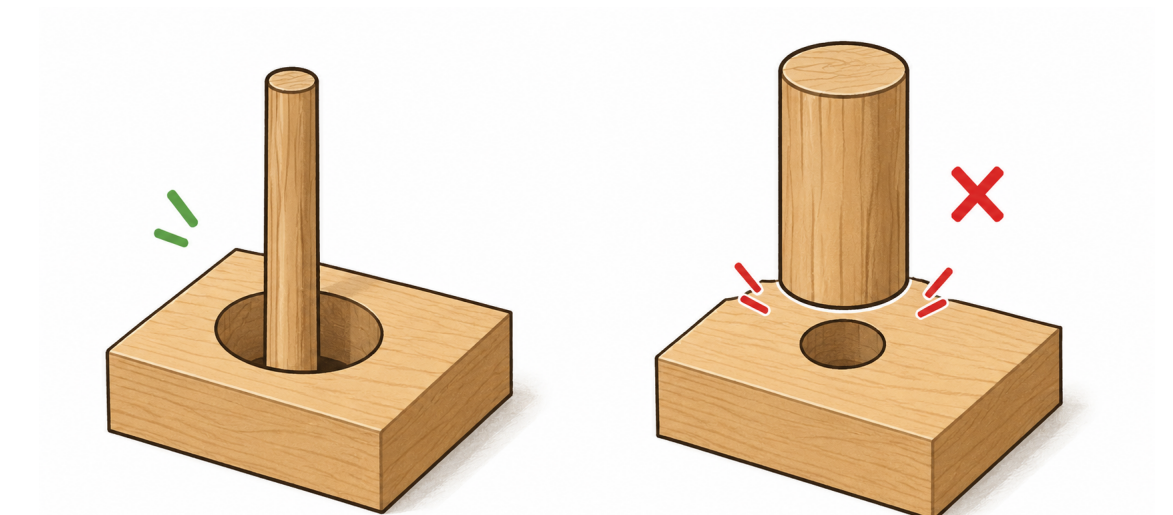
Subtype (2)

- Primitive types, reflexivity (반사성), transitivity (연쇄성)

$$\frac{}{\text{nat} <: \text{int}} \quad \frac{}{\tau <: \tau} \quad \frac{\tau_1 <: \tau_2 \quad \tau_2 <: \tau_3}{\tau_1 <: \tau_3}$$

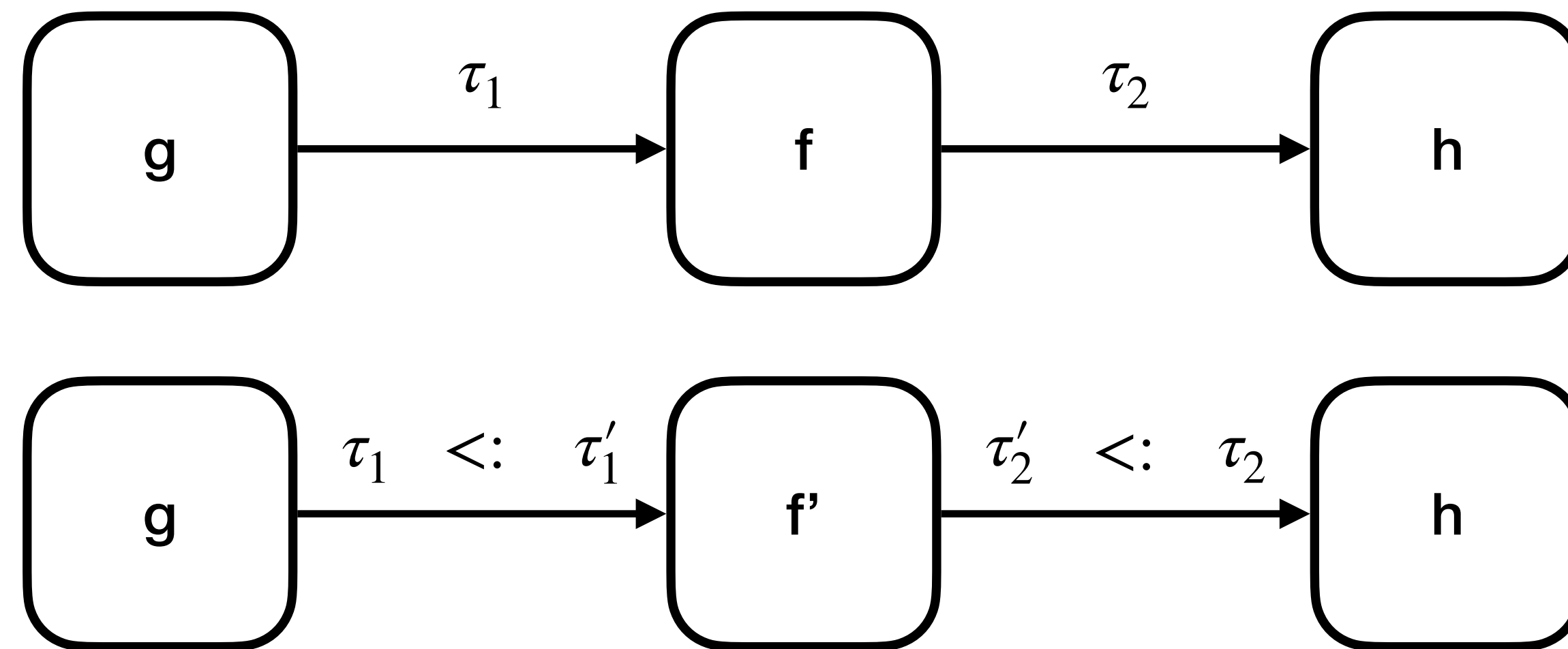
- Function
 - Argument type reversed (contravariant, 거꾸로), return type preserved (covariant, 그대로)
 - Why?

$$\frac{\tau_1 <: \tau'_1 \quad \tau'_2 <: \tau_2}{\tau'_1 \rightarrow \tau'_2 <: \tau_1 \rightarrow \tau_2}$$



Subtype for Functions

- $g : \tau_1 \rightarrow \tau_1, h : \tau_2 \rightarrow \tau_2, f : \tau_1 \rightarrow \tau_2$
- What functions can safely replace f ?
- What types are smaller than $\tau_1 \rightarrow \tau_2$?



$$\frac{\tau_1 <: \tau'_1 \quad \tau'_2 <: \tau_2}{\tau'_1 \rightarrow \tau'_2 <: \tau_1 \rightarrow \tau_2}$$

Subtype (3)

- Record: width, depth, permutation rules

$$\frac{\text{for each } i \quad \sigma_i <: \tau_i}{\{1_i : \sigma_i^{i \in 1 \dots n+k}\} <: \{1_i : \tau_i^{i \in 1 \dots n}\}}$$

$$\frac{\text{for each } i \quad \sigma_i <: \tau_i}{\{1_i : \sigma_i^{i \in 1 \dots n+k}\} <: \{1_i : \tau_i^{i \in 1 \dots n}\}}$$

$$\frac{\{k_j : \sigma_j^{j \in 1 \dots n}\} \text{ is a permutation of } \{1_i : \tau_i^{i \in 1 \dots n}\}}{\{k_j : \sigma_j^{j \in 1 \dots n}\} <: \{1_i : \tau_i^{i \in 1 \dots n}\}}$$

Record Subtyping in OCaml

```
class person (name : string) (age : int) =
  object
    val name = name
    method name = name
  end

class student (name : string) (age : int) (sid : int) =
  object
    inherit person name age (* student <: person *)
    val sid = sid
    method sid = sid
  end

let print_name (p : person) : unit = Printf.printf "%s" p#name

let student1 = new student "Alice" 20 12345
let _ = print_name student1
let person1 = (student1 :> person) (* explicit type coercion *)
let _ = print_name person1
```

*OCaml does not have record subtyping, but we can simulate it using objects.

Subtype (4)

- Variant: width, depth, permutation rules

$$\frac{\text{for each } i \quad \sigma_i <: \tau_i}{[\mathbf{l}_i : \sigma_i^{i \in 1 \dots n}] <: [\mathbf{l}_i : \tau_i^{i \in 1 \dots n+k}]}$$

$$\frac{\text{for each } i \quad \sigma_i <: \tau_i}{[\mathbf{l}_i : \sigma_i^{i \in 1 \dots n+k}] <: [\mathbf{l}_i : \tau_i^{i \in 1 \dots n}]}$$

$$\frac{[\mathbf{k}_j : \sigma_j^{j \in 1 \dots n}] \text{ is a permutation of } [\mathbf{l}_i : \tau_i^{i \in 1 \dots n}]}{[\mathbf{k}_j : \sigma_j^{j \in 1 \dots n}] <: [\mathbf{l}_i : \tau_i^{i \in 1 \dots n}]}$$

Variant Subtyping in OCaml

```
(* num <: const *)
type num = [ `Int of int | `Float of float ]
type const = [ num | `String of string | `Bool of bool ]

let n : num = `Int 42
let c : const = (n :> const) (* explicit type coercion *)

let print_const (c : const) : unit =
  match c with
  | `Int i -> Printf.printf "Int: %d" i
  | `Float f -> Printf.printf "Float: %f" f
  | `String s -> Printf.printf "String: %s" s
  | `Bool b -> Printf.printf "Bool: %b" b

let print_num (n : num) : unit = print_const (n :> const)
```

Exercise

- Assume $\text{Student} <: \text{Person}$
- $\lambda s : \text{Student}. s : \text{Student} \rightarrow \text{Person}$
- $\lambda f : \text{Person} \rightarrow \text{Student}. \lambda s : \text{Student}. f\ s : (\text{Person} \rightarrow \text{Student}) \rightarrow \text{Student} \rightarrow \text{Person}$

Summary

- Subtyping: order relation between types
 - Principle: Less is more (과유불급)
- Foundations of OOP
- A way to support generic programming: one code many uses
- Applications:
 - Subtype polymorphism in OOP
 - Polymorphic variant type in FP